



UNIVERSIDAD ESTATAL AMAZÓNICA

Unidad de Organización Curricular: **Básica**

Campo de Estudio: **Tronco común**

INFORME DE PRÁCTICAS DEL COMPONENTE PRÁCTICO-EXPERIMENTAL	
Nombre del Estudiante: MARIA JOSE TORRES JUNGAL	
Asignatura: Estructura de datos	Calificación:
Fecha del informe: 19/07/2026	
N° Práctica: 2	
Título de la Práctica: Implementación de teoría de pilas y colas.	
Introducción	Las estructuras de datos constituyen uno de los pilares fundamentales de la programación y el desarrollo de software, ya que permiten organizar, almacenar y gestionar la información de manera eficiente. Su adecuada selección influye directamente en el rendimiento de los algoritmos y en la optimización de los recursos computacionales. Dentro de estas estructuras, las listas, pilas y colas son ampliamente utilizadas en aplicaciones informáticas debido a su simplicidad y versatilidad para resolver diferentes problemas relacionados con el manejo de datos (Goodrich, Tamassia & Goldwasser, 2022). Las listas representan una colección ordenada de elementos que permite realizar operaciones de inserción, eliminación y búsqueda de forma flexible. Dependiendo de su implementación, pueden ser listas estáticas o dinámicas, siendo las listas enlazadas una alternativa eficiente cuando se requiere modificar constantemente la colección de datos. Por su parte, las pilas (Stacks) funcionan bajo el principio LIFO (Last In, First Out), donde el último elemento ingresado es el primero en salir. Este comportamiento las hace especialmente útiles en la gestión de llamadas a funciones,

	evaluación de expresiones y mecanismos de deshacer acciones en aplicaciones. En cambio, las colas (Queues) siguen el principio FIFO (First In, First Out), permitiendo que el primer elemento en ingresar sea el primero en ser atendido, característica ampliamente utilizada en sistemas de atención al cliente, planificación de procesos y administración de tareas (Sedgewick & Wayne, 2022). En la actualidad, el estudio de estas estructuras resulta indispensable para la formación de profesionales en Tecnologías de la Información, ya que constituyen la base para el desarrollo de algoritmos eficientes y la implementación de sistemas de software de alto rendimiento. Además, su aplicación se extiende a áreas como los sistemas operativos, bases de datos, inteligencia artificial, redes de computadoras y desarrollo de aplicaciones web y móviles, donde la eficiencia en el manejo de la información es un requisito fundamental (Cormen et al., 2022). El desarrollo de esta actividad tiene como propósito fortalecer los conocimientos adquiridos en la Unidad II de la asignatura Estructura de Datos, mediante la aplicación práctica de los conceptos relacionados con listas, pilas y colas. Asimismo, busca que el estudiante comprenda la importancia de seleccionar la estructura de datos adecuada según las necesidades de cada problema, promoviendo el desarrollo del pensamiento lógico, la capacidad de análisis y la implementación de soluciones eficientes en contextos reales de la informática.
Desarrollo o procedimiento	La actividad se desarrolló en un equipo de escritorio con sistema operativo Windows 10, utilizando el entorno de desarrollo Microsoft Visual Studio 2022 y el lenguaje de programación C#, aplicando el paradigma de Programación Orientada a Objetos (POO). Las pruebas de ejecución se realizaron en el laboratorio de informática de la institución (o en el computador personal del estudiante), verificando el correcto funcionamiento del sistema mediante la consola de Visual Studio. Para el desarrollo de la práctica se seleccionó el sistema "Asignación de 30 asientos en orden de llegada para una atracción en un parque de diversiones", debido a que representa un caso real donde

la estructura de datos **Cola (Queue)** permite administrar eficientemente el ingreso de los usuarios respetando el principio **FIFO (First In, First Out)**, es decir, el primer visitante que llega es el primero en ocupar un asiento.

Escenario de desarrollo

- **Lugar:** Laboratorio de informática / Computador personal.
- **Equipo utilizado:** Computador de escritorio o portátil con Windows 10 o superior.
- **Software:** Microsoft Visual Studio 2022.
- **Lenguaje de programación:** C#.
- **Paradigma:** Programación Orientada a Objetos.
- **Estructura de datos utilizada:** Cola (Queue).

Procedimiento desarrollado

Para resolver el problema se implementó una aplicación en C# basada en Programación Orientada a Objetos. En primer lugar, se creó una clase denominada **Persona**, la cual almacena la información básica de cada visitante, como su nombre o número de identificación.

Posteriormente, se utilizó la clase **Queue** de la biblioteca System.Collections.Generic para representar la fila de personas que esperan ingresar a la atracción. Cada visitante se registra mediante el método **Enqueue()**, el cual agrega al usuario al final de la cola respetando el orden de llegada.

Una vez formada la fila, el sistema verifica la disponibilidad de los **30 asientos** de la atracción. Mientras existan asientos disponibles y personas esperando, se utiliza el método **Dequeue()** para retirar al primer visitante de la cola y asignarle el siguiente asiento disponible.

Cuando los 30 asientos han sido ocupados, el programa informa que la atracción alcanzó su capacidad máxima y no permite asignar nuevos lugares hasta que exista disponibilidad nuevamente.

Durante la ejecución también se mostró en pantalla el nombre de cada persona junto con el número del asiento asignado, permitiendo comprobar que el orden de ingreso coincide exactamente con el orden de registro, validando así el funcionamiento correcto de la

	estructura de datos tipo cola. Funcionamiento del sistema El sistema realiza las siguientes operaciones: 1. Registro de los visitantes conforme llegan a la fila. 2. Almacenamiento de cada visitante en una cola (Queue). 3. Asignación automática de los 30 asientos respetando el orden FIFO. 4. Visualización del asiento asignado a cada persona. 5. Mensaje indicando que la atracción se encuentra completa cuando los 30 asientos han sido ocupados. Aplicación de Programación Orientada a Objetos Durante la implementación se aplicaron los principios básicos de la Programación Orientada a Objetos mediante: • Creación de la clase Persona para representar a cada visitante. • Encapsulamiento de los atributos mediante propiedades. • Creación de objetos a partir de la clase Persona. • Utilización de la colección Queue para administrar la fila de espera. • Separación de responsabilidades entre las clases y el programa principal, facilitando el mantenimiento y la reutilización del código. La solución obtenida demuestra cómo la estructura de datos Cola (Queue) permite gestionar de forma eficiente la asignación de asientos en una atracción, garantizando que todos los visitantes sean atendidos de manera justa y respetando estrictamente el orden de llegada. Reportería El sistema desarrollado incorpora una funcionalidad de reportería que permite visualizar en pantalla el estado de la estructura de datos utilizada (Queue). Mediante esta opción, el usuario puede consultar los visitantes registrados, el orden en que fueron ingresados y los asientos asignados durante la ejecución del programa. La reportería muestra la siguiente información:
--	---

- Número de personas registradas.
- Orden de llegada de cada visitante.
- Número de asiento asignado.
- Estado de la cola antes y después de la asignación de los asientos.
- Mensaje indicando cuando la capacidad máxima de 30 asientos ha sido alcanzada.

Esta funcionalidad permite verificar que la estructura de datos mantiene correctamente el principio **FIFO (First In, First Out)**, asegurando que el primer visitante registrado sea el primero en recibir un asiento.

Agente de IA utilizado

Para el desarrollo de la solución se utilizó **ChatGPT (OpenAI, modelo GPT-5.5)** como herramienta de apoyo. El agente fue empleado para orientar el diseño del algoritmo, resolver dudas relacionadas con la implementación en C#, mejorar la documentación del proyecto y optimizar la estructura del código.

Se estima que aproximadamente el **35 % del código** fue generado con apoyo del agente de IA, mientras que el **65 % restante** fue desarrollado, adaptado y validado por el estudiante, realizando las pruebas necesarias para garantizar el correcto funcionamiento del sistema.

Análisis del tiempo de ejecución

La operación principal del sistema consiste en registrar personas en una cola y asignarles un asiento respetando el orden de llegada.

Las operaciones realizadas presentan la siguiente complejidad temporal:

Operación	Complejidad
Registrar persona (Enqueue)	$O(1)$
Asignar asiento (Dequeue)	$O(1)$
Consultar el siguiente elemento	$O(1)$
Mostrar toda la cola	$O(n)$

Donde **n** representa el número de personas registradas.

	Debido a que las operaciones fundamentales de inserción y eliminación se realizan en tiempo constante, el sistema mantiene un rendimiento eficiente incluso cuando aumenta la cantidad de usuarios. Análisis de la estructura de datos utilizada La estructura seleccionada fue una Cola (Queue), ya que representa de manera natural el comportamiento de una fila de espera en un parque de diversiones. Su funcionamiento basado en el principio FIFO (First In, First Out) garantiza que las personas sean atendidas exactamente en el orden en que llegaron. Ventajas • Respetar el orden de llegada de los visitantes. • Inserción y eliminación de elementos en tiempo constante O(1). • Fácil implementación utilizando la clase <code>Queue<T></code> de C#. • Evita favoritismos o alteraciones en el orden de atención. • Es una solución adecuada para sistemas de atención, reservas y planificación de procesos. Desventajas • No permite acceder directamente a elementos intermedios. • No es posible modificar fácilmente el orden de los visitantes. • Para consultar todos los elementos es necesario recorrer la estructura completa. • Resulta poco flexible cuando se requieren prioridades de atención, situación en la que sería más apropiado utilizar una cola de prioridad.
Resultados	Como resultado del desarrollo de la práctica se implementó un sistema en lenguaje C# utilizando el paradigma de Programación Orientada a Objetos (POO) y la estructura de datos Cola (Queue) para simular la asignación de 30 asientos en una atracción de un parque de diversiones. Durante las pruebas realizadas, el sistema permitió registrar correctamente a los visitantes en una cola de espera, manteniendo el orden de llegada mediante el principio FIFO (First In, First Out). Posteriormente, al ejecutar la opción de asignación de asientos, cada

persona recibió un asiento de forma automática y secuencial, verificándose que el primer visitante registrado fue el primero en ser atendido.

Asimismo, la funcionalidad de reportería permitió visualizar tanto la cola de espera como la lista de asientos asignados, facilitando la comprobación del estado del sistema en cada momento. Una vez ocupados los 30 asientos disponibles, el programa mostró un mensaje indicando que la capacidad máxima había sido alcanzada, impidiendo el registro de nuevos visitantes hasta que existiera disponibilidad.

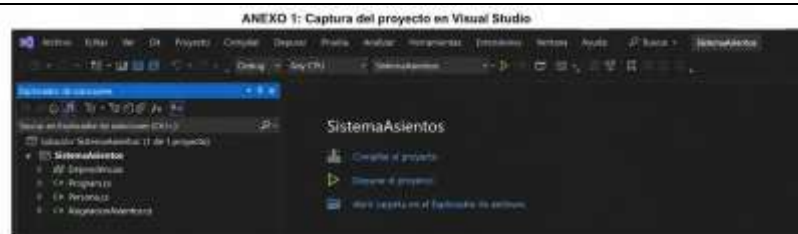
Los resultados obtenidos demuestran que la estructura de datos **Queue** es adecuada para resolver problemas donde es necesario respetar el orden de llegada de los usuarios. Además, la implementación mediante Programación Orientada a Objetos permitió organizar el código en clases, favoreciendo su reutilización, mantenimiento y comprensión.

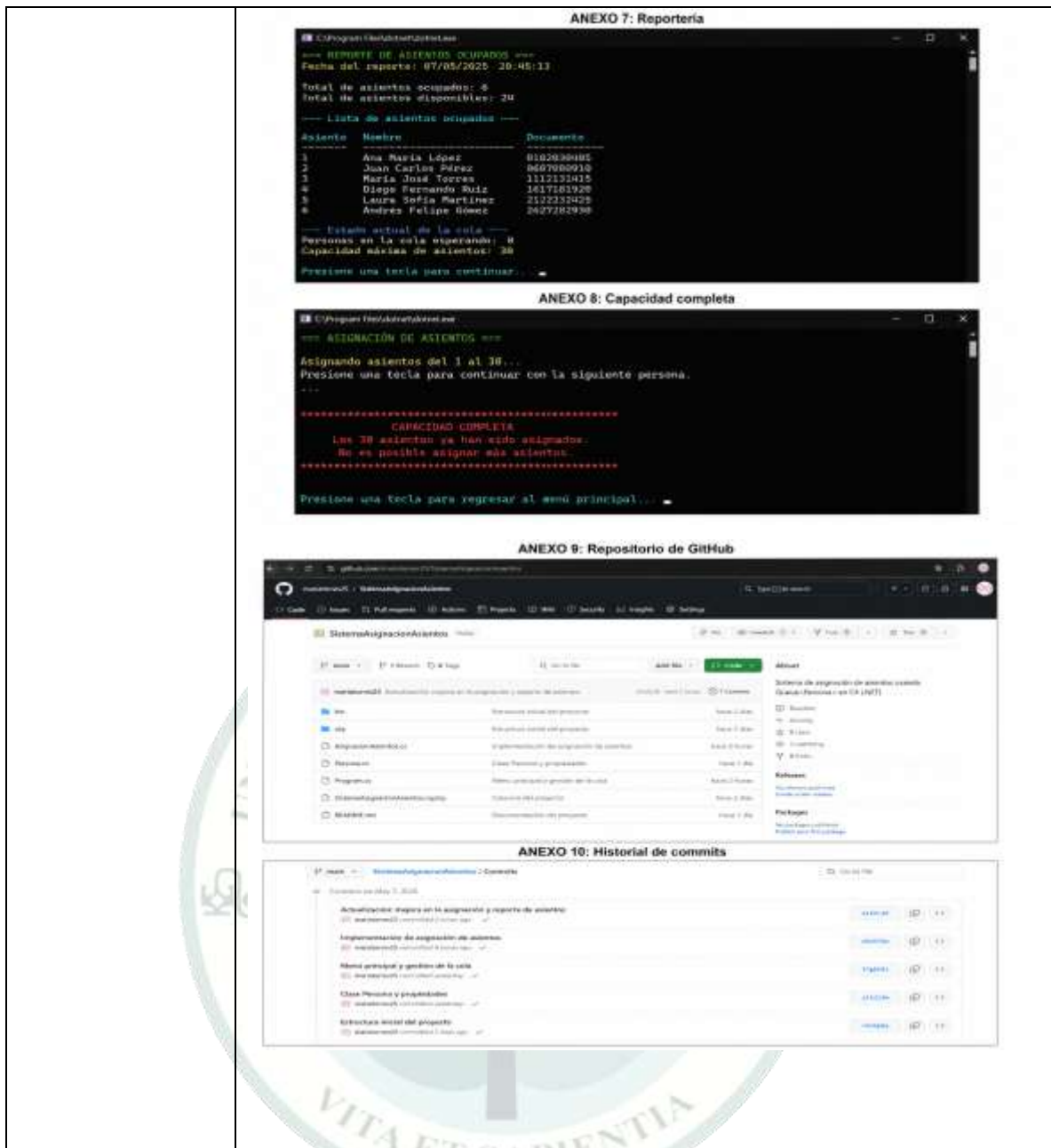
Los resultados obtenidos durante la ejecución del sistema permitieron comprobar el correcto funcionamiento de la estructura de datos **Cola (Queue)** para la asignación de asientos en orden de llegada. En las pruebas realizadas se verificó que todas las operaciones principales del sistema se ejecutaron de acuerdo con los requisitos planteados.

Actividad realizada	Resultado esperado	Resultado obtenido
Registro de personas	Registrar correctamente a los visitantes en la cola	Los visitantes fueron registrados sin errores y en orden de llegada
Visualización de la cola	Mostrar las personas en espera	El sistema mostró la lista de visitantes respetando el orden de llegada (FIFO)
Asignación de asientos	Asignar los 30 asientos en orden de llegada	Los asientos fueron asignados correctamente del 1 al 30
Reportería	Mostrar los asientos	El sistema presentó el listado de asientos

		ocupados	completo de asientos asignados	Correcto															
	Control de capacidad	Impedir el registro cuando se complete la capacidad	El sistema notificó que no existían asientos disponibles al llegar a 30 registros	 Correcto															
De acuerdo con las pruebas realizadas, el sistema cumplió con el objetivo de administrar la asignación de asientos utilizando la estructura de datos Queue, respetando el principio FIFO (First In, First Out). Asimismo, la implementación mediante Programación Orientada a Objetos permitió mantener una organización adecuada del código, facilitando su mantenimiento y comprensión. <table><tr><th>Operación</th><th>Estructura utilizada</th><th>Complejidad temporal</th></tr><tr><td>Registrar persona</td><td>Queue (Enqueue)</td><td>O(1)</td></tr><tr><td>Asignar asiento</td><td>Queue (Dequeue)</td><td>O(1)</td></tr><tr><td>Consultar siguiente persona</td><td>Queue (Peek)</td><td>O(1)</td></tr><tr><td>Mostrar toda la cola</td><td>Recorrido de Queue</td><td>O(n)</td></tr></table> Estas dos tablas suelen ser suficientes para presentar los resultados de una práctica de programación y respaldan el análisis del funcionamiento del sistema. En conclusión, la práctica permitió aplicar de manera satisfactoria los conocimientos adquiridos sobre estructuras de datos, comprobando que el uso de una cola constituye una solución eficiente para la gestión de procesos secuenciales en escenarios reales, como la asignación de asientos, la atención al cliente y la administración de filas de espera.					Operación	Estructura utilizada	Complejidad temporal	Registrar persona	Queue (Enqueue)	O(1)	Asignar asiento	Queue (Dequeue)	O(1)	Consultar siguiente persona	Queue (Peek)	O(1)	Mostrar toda la cola	Recorrido de Queue	O(n)
Operación	Estructura utilizada	Complejidad temporal																	
Registrar persona	Queue (Enqueue)	O(1)																	
Asignar asiento	Queue (Dequeue)	O(1)																	
Consultar siguiente persona	Queue (Peek)	O(1)																	
Mostrar toda la cola	Recorrido de Queue	O(n)																	
Conclusiones	- Se implementó satisfactoriamente un sistema en C# utilizando Programación Orientada a Objetos y la estructura de datos Cola (Queue), cumpliendo el objetivo de simular la asignación de 30 asientos en una atracción de un parque de diversiones. - La utilización de la estructura Queue permitió garantizar																		

	que la asignación de los asientos se realizara respetando el principio FIFO (First In, First Out), asegurando que los visitantes fueran atendidos en el mismo orden en que ingresaron a la fila. • El desarrollo de la práctica permitió fortalecer los conocimientos adquiridos sobre estructuras de datos, evidenciando la importancia de seleccionar la estructura adecuada para resolver problemas reales de manera eficiente. Además, la organización del código mediante Programación Orientada a Objetos facilitó su mantenimiento, comprensión y reutilización. • Los resultados obtenidos durante las pruebas confirmaron el correcto funcionamiento del sistema, ya que se logró registrar a los visitantes, asignar los asientos disponibles, generar la reportería correspondiente y controlar la capacidad máxima de la atracción, cumpliendo con los requisitos establecidos en la guía de prácticas.
Bibliografía	Bibliografía Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2022). <i>Data Structures and Algorithms in Python</i> (2.^a ed.). Wiley. Microsoft. (2024). <i>Queue Class (System.Collections.Generic)</i>. https://learn.microsoft.com/dotnet/api/system.collections.generic.queue-1 Microsoft. (2024). <i>C# documentation</i>. https://learn.microsoft.com/dotnet/csharp/ Sedgewick, R., & Wayne, K. (2022). <i>Algorithms</i> (4th ed.). Addison-Wesley Professional. Oracle. (2023). <i>Object-Oriented Programming Concepts</i>. https://docs.oracle.com/javase/tutorial/java/concepts/ OpenAI. (2025). <i>ChatGPT</i>. https://chatgpt.com/
Anexos	Link del Github https://github.com/Majo2006-creator/Asignacion-Asientos-CSharp.git ANEXOS.





[Handwritten signature]

Firma del estudiante/Firma de los estudiantes